

# Contents

|          |                               |          |
|----------|-------------------------------|----------|
| <b>1</b> | <b>Introduction</b>           | <b>3</b> |
| 1.1      | Motivation and Goals          | 3        |
|          | Why Distributed?              | 3        |
|          | Key Challenges                | 3        |
| 1.2      | Historical Context            | 3        |
| <b>2</b> | <b>System Models</b>          | <b>3</b> |
| 2.1      | Synchronous vs. Asynchronous  | 3        |
| 2.2      | Failure Models                | 3        |
|          | Crash Failures                | 4        |
|          | Byzantine Failures            | 4        |
| <b>3</b> | <b>The Consensus Problem</b>  | <b>6</b> |
| 3.1      | FLP Impossibility             | 6        |
| 3.2      | Paxos                         | 6        |
|          | Basic Paxos                   | 6        |
|          | Multi-Paxos                   | 6        |
| <b>4</b> | <b>Raft</b>                   | <b>6</b> |
| 4.1      | Leader Election               | 6        |
| 4.2      | Log Replication               | 7        |
| <b>5</b> | <b>Replication Strategies</b> | <b>9</b> |
| 5.1      | Leader-Based Replication      | 9        |
| 5.2      | Leaderless Replication        | 9        |
| <b>6</b> | <b>Failure Detection</b>      | <b>9</b> |
| 6.1      | Heartbeat Mechanisms          | 9        |
| 6.2      | Phi Accrual Failure Detector  | 9        |



# Introduction

Distributed systems form the backbone of modern computing infrastructure. From cloud services to social networks, the ability to coordinate multiple autonomous machines is fundamental to scalability and reliability.

## 1.1 Motivation and Goals

The primary motivation for studying distributed systems lies in their ubiquity. Every time you use a web application, stream a video, or check your email, you are interacting with a distributed system.

- Why Distributed?

Centralized systems, while simpler to reason about, face inherent limitations in scale, fault tolerance, and geographic distribution. A single point of failure can bring down an entire service.

- Key Challenges

Consistency, availability, and partition tolerance form the well-known CAP theorem triangle, forcing system designers to make deliberate trade-offs.

## 1.2 Historical Context

The field has evolved from early remote procedure calls in the 1980s through the rise of the internet, to modern microservices architectures and serverless computing paradigms.

# System Models

Formal models provide the theoretical foundation for reasoning about distributed systems. These models abstract away implementation details to focus on essential properties.

## 2.1 Synchronous vs. Asynchronous

In synchronous models, message delivery is bounded by a known delay. Asynchronous models make no such guarantees, making them more realistic but harder to reason about.

## 2.2 Failure Models

Understanding different types of failures—crash, omission, Byzantine—is essential for designing fault-tolerant systems that can withstand adversarial conditions.

- **Crash Failures**

A crash failure occurs when a process halts prematurely and never recovers. This is the simplest failure model and forms the basis for most practical fault-tolerance mechanisms.

- **Byzantine Failures**

Byzantine failures are the most general and most difficult to handle. A Byzantine process may send contradictory information to different parts of the system, making consensus especially challenging.



## The Consensus Problem

At the heart of every distributed system lies the challenge of getting independent nodes to agree on a single value or course of action, despite communication delays and potential failures.

### 3.1 FLP Impossibility

The seminal result by Fischer, Lynch, and Paterson (1985) proved that deterministic consensus is impossible in purely asynchronous systems with even a single failure. This impossibility result has shaped decades of research.

### 3.2 Paxos

Leslie Lamport's Paxos protocol is one of the most influential consensus algorithms. Despite its reputation for being difficult to understand, its core ideas are surprisingly elegant.

- Basic Paxos

Basic Paxos guarantees consensus among a set of processes, provided a majority are responsive. It operates in two phases: prepare and accept, using proposal numbers to ensure safety.

- Multi-Paxos

Multi-Paxos optimizes the basic protocol for sequences of decisions by electing a stable leader, eliminating the prepare phase for subsequent values and significantly improving throughput.

## Raft

Raft was designed as an understandable alternative to Paxos, decomposing the consensus problem into independent subproblems: leader election, log replication, and safety.

### 4.1 Leader Election

Raft uses randomized timeouts to trigger leader elections. When a follower does not receive a heartbeat within its election timeout, it becomes a candidate and requests votes from its peers.

## **4.2** Log Replication

Once a leader is established, clients send commands to the leader, which appends them to its log and replicates them across followers using `AppendEntries` RPCs.





## Replication Strategies

Replication is the primary mechanism for achieving fault tolerance in distributed systems. Different strategies offer varying trade-offs between consistency, performance, and complexity.

### 5.1 Leader-Based Replication

In leader-based replication, all writes go through a single leader node that propagates changes to followers. This simplifies consistency but introduces a single point of coordination.

### 5.2 Leaderless Replication

Leaderless (or quorum-based) replication allows any node to accept writes, using quorums to ensure consistency. Dynamo-style systems use this approach for high availability.

## Failure Detection

Detecting failures accurately and quickly is crucial for maintaining system health. However, distinguishing between a failed process and a temporarily slow network is notoriously difficult.

### 6.1 Heartbeat Mechanisms

Most practical systems use heartbeat-based failure detection, where nodes periodically exchange messages. Absence of a heartbeat within a timeout triggers a failure suspicion.

### 6.2 Phi Accrual Failure Detector

The phi accrual failure detector improves upon simple timeouts by maintaining a probabilistic model of network delays, producing a continuous suspicion level rather than a binary alive/dead verdict.